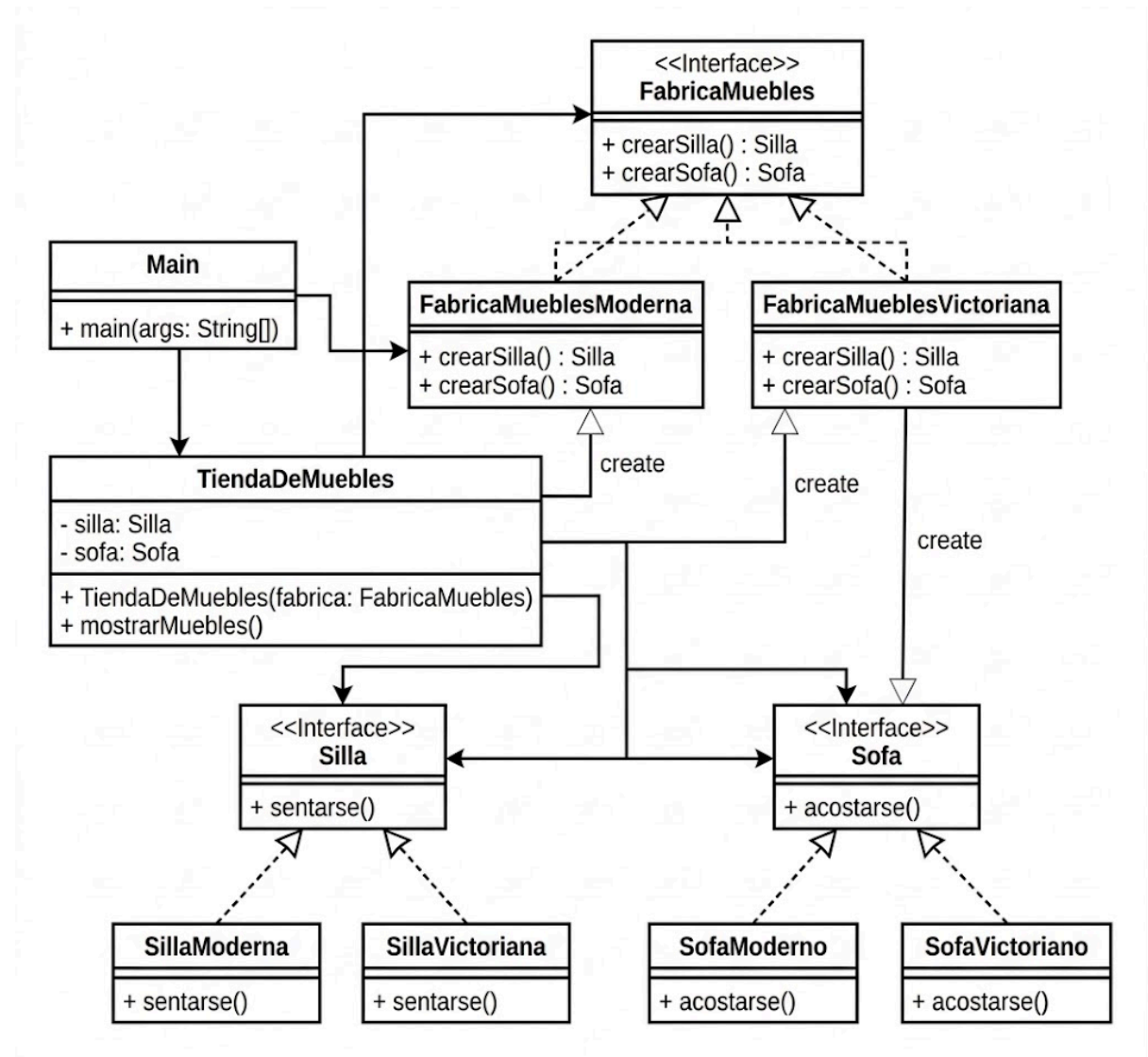


Trabajo Práctico: Patrones de Diseño

1. Diagrama UML de Clases:

a. Elaborar el diagrama UML de clases correspondiente al patrón **Abstract Factory**.



b. Identificar y describir las clases, interfaces, atributos, métodos y relaciones involucradas.

El patrón **Abstract Factory** permite producir familias de objetos relacionados (en este caso, muebles de estilo *Moderno* o *Victoriano*) sin especificar sus clases concretas. Los elementos involucrados en el código son los siguientes:

1. Interfaces (Productos Abstractos y Fábrica Abstracta)

- **FabricaMuebles (Interfaz / Fábrica Abstracta):**
 - *Descripción:* Declara los métodos de creación para cada producto de la familia de muebles.
 - *Métodos:*
 - crearSilla(): Silla
 - crearSofa(): Sofa
- **Silla (Interfaz / Producto Abstracto A):**
 - *Descripción:* Define el contrato común para todas las variantes de sillas.
 - *Métodos:*
 - sentarse(): void
- **Sofa (Interfaz / Producto Abstracto B):**
 - *Descripción:* Define el contrato común para todas las variantes de sofás.
 - *Métodos:*
 - acostarse(): void

2. Clases Concretas (Fábricas Concretas)

- **FabricaMueblesModerna:**
 - *Descripción:* Implementa la interfaz FabricaMuebles para instanciar la familia de productos modernos.
 - *Métodos:*
 - crearSilla(): Silla (Retorna una nueva instancia de SillaModerna).
 - crearSofa(): Sofa (Retorna una nueva instancia de SofaModerno).
- **FabricaMueblesVictoriana:**
 - *Descripción:* Implementa la interfaz FabricaMuebles para instanciar la familia de productos victorianos.
 - *Métodos:*
 - crearSilla(): Silla (Retorna una nueva instancia de SillaVictoriana).
 - crearSofa(): Sofa (Retorna una nueva instancia de SofaVictoriano).

3. Clases Concretas (Productos Concretos)

- **SillaModerna:** Implementa la interfaz Silla. Define el comportamiento

- específico de una silla moderna. Método: `sentarse()`.
- `SillaVictoriana`: Implementa la interfaz `Silla`. Define el comportamiento específico de una silla victoriana. Método: `sentarse()`.

-
- SofaModerno: Implementa la interfaz Sofa. Define el comportamiento específico de un sofá moderno. Método: acostarse().
- SofaVictoriano: Implementa la interfaz Sofa. Define el comportamiento específico de un sofá victoriano. Método: acostarse().

4. Clases Cliente y de Ejecución

- TiendaDeMuebles: Clase cliente que recibe una FabricaMuebles por inyección y utiliza sus métodos abstractos para crear y mostrar los muebles sin preocuparse por las clases concretas.
- Main: Clase de entrada que inicializa las fábricas concretas (FabricaMueblesModerna o FabricaMueblesVictoriana) y se las pasa a la tienda.

5. Relaciones Involucradas

- Implementación (implements):
 - FabricaMueblesModerna y FabricaMueblesVictoriana implementan la interfaz FabricaMuebles.
 - SillaModerna y SillaVictoriana implementan la interfaz Silla.
 - SofaModerno y SofaVictoriano implementan la interfaz Sofa.
- Asociación / Dependencia (Creación): Las fábricas concretas crean objetos de los productos concretos correspondientes a su familia garantizando que combinen entre sí (ej. FabricaMueblesModerna crea SillaModerna y SofaModerno).

2. Análisis del Patrón Para cada uno de los patrones seleccionados, explicar:

a. Problema

El problema aparece cuando necesitamos crear varios objetos relacionados entre sí, pero queremos que todos pertenezcan a la misma familia o estilo.

En este caso, la tienda necesita crear muebles como Silla y Sofa, pero existen diferentes estilos: moderno y victoriano. Si la tienda tuviera que decidir directamente qué clase concreta crear (SillaModerna, SofaModerno, etc.), quedaría acoplada a esas clases y sería más difícil agregar nuevos estilos.

Además, existe el riesgo de combinar productos de diferentes familias, por ejemplo, crear una SillaModerna junto con un SofaVictoriano.

b. Solución

Abstract Factory propone crear una fábrica abstracta que define los métodos necesarios para crear cada producto de una familia.

En este caso, FabricaMuebles define:

- crearSilla()
- crearSofa()

Luego, cada fábrica concreta crea una familia completa de productos:

- FabricaMueblesModerna → SillaModerna y SofaModerno.
- FabricaMueblesVictoriana → SillaVictoriana y SofaVictoriano.

La clase TiendaDeMuebles trabaja solamente con las interfaces FabricaMuebles, Silla y Sofa. Por lo tanto, no necesita conocer las clases concretas.

De esta manera, al cambiar la fábrica que recibe la tienda, se puede cambiar toda la familia de muebles sin modificar la lógica de la tienda.

c. Consecuencias

Ventajas:

- Permite crear familias completas de objetos relacionados.
- Evita mezclar productos de diferentes familias.
- Reduce el acoplamiento entre el cliente y las clases concretas.
- Facilita agregar una nueva familia de productos sin modificar el cliente.
- Centraliza la creación de los objetos en las fábricas concretas.

Desventajas:

- Aumenta la cantidad de clases e interfaces necesarias.
- Agregar un nuevo tipo de producto, por ejemplo Mesa, requiere modificar la fábrica abstracta y todas las fábricas concretas.
- Puede resultar más complejo que crear directamente los objetos cuando el sistema es pequeño.

3. Implementación:

a. Desarrollar un ejemplo de implementación en código para cada patrón de diseño.

Desarrollado en GitHub.

b. Incluir comentarios y una breve explicación del funcionamiento de la solución implementada.

Explicación:

La implementación utiliza el patrón Abstract Factory para crear familias de muebles que pertenecen al mismo estilo. La interfaz `FabricaMuebles` define los métodos para crear una silla y un sofá, y cada fábrica concreta se encarga de crear los muebles de un estilo determinado.

Por ejemplo, `FabricaMueblesModerna` crea una `SillaModerna` y un `SofaModerno`, mientras que `FabricaMueblesVictoriana` crea una `SillaVictoriana` y un `SofaVictoriano`.

La clase `TiendaDeMuebles` recibe una fábrica y utiliza sus métodos para obtener los muebles, sin tener que conocer qué clases concretas se están creando. Finalmente, en `Main` se elige qué fábrica utilizar según el estilo que se quiera, permitiendo cambiar toda la familia de muebles sin modificar la tienda.